

基于 NFSR 的分组密码算法 SPRING^{*}

田甜, 戚文峰, 叶晨东, 谢晓锋

战略支援部队信息工程大学, 郑州 450000

通讯作者: 田甜, E-mail: tiantian_d@126.com

摘要: 本文详细介绍分组密码算法 SPRING 的设计. 根据分组长度和密钥长度的不同, SPRING 具体包括三个算法版本 SPRING-128-128、SPRING-128-256、SPRING-256-256, 其中 SPRING- n - m 表示分组长度为 n 且密钥长度为 m . SPRING 是 SP 结构的分组密码, 主要面向硬件实现设计, 采用基于非线性反馈移位寄存器 (NFSR) 的 32-比特 S-盒和基于非线性反馈移位寄存器的密钥扩展算法. 由于没有 S-盒的存储, SPRING 算法的硬件实现面积比较小. 根据不同的应用需求, 例如面积小或加/解密速率高, 可以采用不同的实现方式. 基于轮的实现, 硬件面积最小, 在 TSMC 16 nm 工艺库下, SPRING-128-128 算法的硬件实现面积约 $1046\ \mu\text{m}^2$; 全轮展开实现, 加/解密速率最大, SPRING-128-128 算法的加密速率可以达到 17 482 Mbps, 此时硬件实现面积约 $8079\ \mu\text{m}^2$. SPRING 的含义为 SP 结构分组密码和环状串联非线性反馈移位寄存器 (A ring-like cascade connection of NFSRs).

关键词: 分组密码; 非线性反馈移位寄存器; 轻量密码; SPRING

中图分类号: TP309.7 **文献标识码:** A **DOI:** 10.13868/j.cnki.jcr.000343

中文引用格式: 田甜, 戚文峰, 叶晨东, 谢晓锋. 基于 NFSR 的分组密码算法 SPRING[J]. 密码学报, 2019, 6(6): 815–834.

英文引用格式: TIAN T, QI W F, YE C D, XIE X F. SPRING: A family of small hardware-oriented block ciphers based on NFSRs[J]. Journal of Cryptologic Research, 2019, 6(6): 815–834.

SPRING: A Family of Small Hardware-oriented Block Ciphers Based on NFSRs

TIAN Tian, QI Wen-Feng, YE Chen-Dong, XIE Xiao-Feng

PLA Strategic Support Force Information Engineering University, Zhengzhou 450000, China

Corresponding author: TIAN Tian, E-mail: tiantian_d@126.com

Abstract: This paper presents a new lightweight block cipher family SPRING. The family contains of three block ciphers, namely SPRING-128-128, SPRING-128-256, and SPRING-256-256, where SPRING- n - m denotes n -bit block size and m -bit key size. SPRING is an SPN cipher which is designed mainly for compact hardware implementation. In particular, SPRING uses nonlinear feedback shift registers (NFSRs) to generate 32-bit S-boxes and all round keys. Since no S-box is stored, the hardware implementation of SPRING has a very low cost. Moreover, there are two implementation strategies

^{*} 基金项目: 国家自然科学基金 (61672533, 61521003)

Foundation: National Natural Science Foundation of China (61672533, 61521003)

收稿日期: 2019-11-29 定稿日期: 2019-12-10

that target at small area and high throughput, respectively. The round-based implementation can be done with a very small hardware area. For example, the area of SPRING-128-128 is only $1046 \mu\text{m}^2$ with TSMC 16 nm. The fully unrolled implementation has high throughput. For example, the throughput of SPRING-128-128 attains 17 482 Mbps with the area $8079 \mu\text{m}^2$ in the fully unrolled implementation. Finally, SPRING means an SPN cipher and a ring-like cascade connection of NFSRs.

Key words: block ciphers; nonlinear feedback shift registers; lightweight encryption; SPRING

1 SPRING 算法概述

SPRING 是基于非线性反馈移位寄存器设计的轻量级分组密码, 采用 SP 结构^[1], 支持的分组长/密钥长度参数为 128/128、128/256 和 256/256. SPRING- n - m 表示分组长度为 n 且密钥长度为 m 的分组密码算法. SPRING-128-128 的迭代轮数为 10 轮, SPRING-128-256 的迭代轮数为 14 轮, SPRING-256-256 的迭代轮数为 18 轮. 为了算法描述的一致性, 以下记算法迭代轮数为 $N_r \in \{10, 14, 18\}$, 由分组长度/密钥长度决定具体取值. SPRING- n - m 系列加密算法和解密算法的设计框架参见算法 1 和算法 2, 其中加密算法 ENCRYPTION 包含密钥扩展算法 KeyExpansion、轮函数 Round、最后一轮轮函数 FinalRound, 解密算法 DECRYPTION 与加密算法具有相似性, 也包括密钥扩展算法 InvKeyExpansion、轮函数 InvRound、最后一轮轮函数 InvFinalRound.

算法 1 ENCRYPTION	算法 2 DECRYPTION
Input: plaintext, key	Input: ciphertext, key
Output: ciphertext	Output: plaintext
1 state $\leftarrow$ plaintext;	1 state $\leftarrow$ ciphertext;
2 $(\text{rk}_1, \text{rk}_2, \dots, \text{rk}_{N_r+1}) = \text{KeyExpansion}(\text{key});$	2 $(\text{rk}_1^{\text{inv}}, \text{rk}_2^{\text{inv}}, \dots, \text{rk}_{N_r+1}^{\text{inv}}) = \text{InvKeyExpansion}(\text{key});$
3 for $1 \leq i \leq N_r - 1$ do	3 for $1 \leq i \leq N_r - 1$ do
4 Round(state, rk_i);	4 InvRound(state, rk_i^{inv});
5 end	5 end
6 FinalRound(state, $\text{rk}_{N_r}, \text{rk}_{N_r+1}$);	6 InvFinalRound(state, $\text{rk}_{N_r}^{\text{inv}}, \text{rk}_{N_r+1}^{\text{inv}}$);
7 ciphertext $\leftarrow$ state;	7 plaintext $\leftarrow$ state;

2 SPRING-128-128 的设计

本节介绍 SPRING-128-128 的具体设计参数.

2.1 SPRING-128-128 的内部状态

SPRING-128-128 的内部状态以字节为单位, 看作 $\mathbb{F}_{2^8}$ 上的 16 维向量或者 $\mathbb{F}_{2^8}$ 上 4×4 的矩阵, 即

$$\begin{aligned}
 \text{state} &= s_0 \parallel s_1 \parallel s_2 \parallel s_3 \parallel s_4 \parallel s_5 \parallel s_6 \parallel s_7 \parallel s_8 \parallel s_9 \parallel s_{10} \parallel s_{11} \parallel s_{12} \parallel s_{13} \parallel s_{14} \parallel s_{15} \\
 &= (s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_{11}, s_{12}, s_{13}, s_{14}, s_{15}) \\
 &= \begin{pmatrix} s_0 & s_1 & s_2 & s_3 \\ s_4 & s_5 & s_6 & s_7 \\ s_8 & s_9 & s_{10} & s_{11} \\ s_{12} & s_{13} & s_{14} & s_{15} \end{pmatrix}
 \end{aligned}$$

其中 s_i 为一个字节, 即 8 比特, $0 \leq i \leq 15$. 以下内部状态矩阵总是记为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix}$$

设 128 比特的明文分组为 $\text{plaintext} = p_0 \parallel p_1 \parallel \cdots \parallel p_{15}$, 其中 p_i 是一个字节, $0 \leq i \leq 15$, 则用明文填充内部状态的规则为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix} \leftarrow \begin{pmatrix} p_0 & p_1 & p_2 & p_3 \\ p_4 & p_5 & p_6 & p_7 \\ p_8 & p_9 & p_{10} & p_{11} \\ p_{12} & p_{13} & p_{14} & p_{15} \end{pmatrix}$$

设一个 128 比特密文分组为 $\text{ciphertext} = c_0 \parallel c_1 \parallel \cdots \parallel c_{15}$, 其中 c_i 是一个字节, $0 \leq i \leq 15$, 则内部状态与输出密文分组的对应关系为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix} \rightarrow \begin{pmatrix} c_0 & c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 & c_7 \\ c_8 & c_9 & c_{10} & c_{11} \\ c_{12} & c_{13} & c_{14} & c_{15} \end{pmatrix}$$

2.2 SPRING-128-128 加密算法

SPRING-128-128 算法的迭代轮数为 10 轮, 即 $N_r = 10$. SPRING-128-128 加密算法轮函数 Round-128 包括三个子函数: AddRoundKey, SubRow 和 Transpose. 在每一轮轮变换中, 首先加轮子密钥, 然后对内部状态矩阵的每一行分别做 SubRow 运算, 最后对内部状态进行一次转置, 参见算法 3.

SPRING-128-128 最后一轮轮函数 FinalRound-128 包括三个子函数: AddRoundKey, SubRow 和 BitPermutation, 其中在 SubRow 运算前后各做了一次 AddRoundKey, 参见算法 4.

算法 3 Round-128	算法 4 FinalRound-128
Input: state, rk_i 1 AddRoundKey(state, rk_i); 2 for $1 \leq j \leq 4$ do 3 SubRow($s_{j1}, s_{j2}, s_{j3}, s_{j4}$); 4 end 5 Transpose(state);	Input: state, $\text{rk}_{N_r}, \text{rk}_{N_r+1}$ 1 AddRoundKey(state, rk_{N_r}); 2 for $1 \leq j \leq 4$ do 3 SubRow($s_{j1}, s_{j2}, s_{j3}, s_{j4}$); 4 end 5 AddRoundKey(state, rk_{N_r+1}); 6 BitPermutation(state);

综合上述定义, SPRING-128-128 加密算法的整体结构参见图 1, 其中 $\text{plaintext} = p_0 \parallel p_1 \parallel \cdots \parallel p_{15}$ 是输入明文分组, $\text{ciphertext} = c_0 \parallel c_1 \parallel \cdots \parallel c_{15}$ 是输出密文分组.

2.2.1 AddRoundKey 的定义

AddRoundKey 完成对内部状态与轮子密钥异或加的功能. 设 AddRoundKey 的输入内部状态为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix}$$

输入轮子密钥为

$$\begin{pmatrix} k_{11} & k_{12} & k_{13} & k_{14} \\ k_{21} & k_{22} & k_{23} & k_{24} \\ k_{31} & k_{32} & k_{33} & k_{34} \\ k_{41} & k_{42} & k_{43} & k_{44} \end{pmatrix}$$

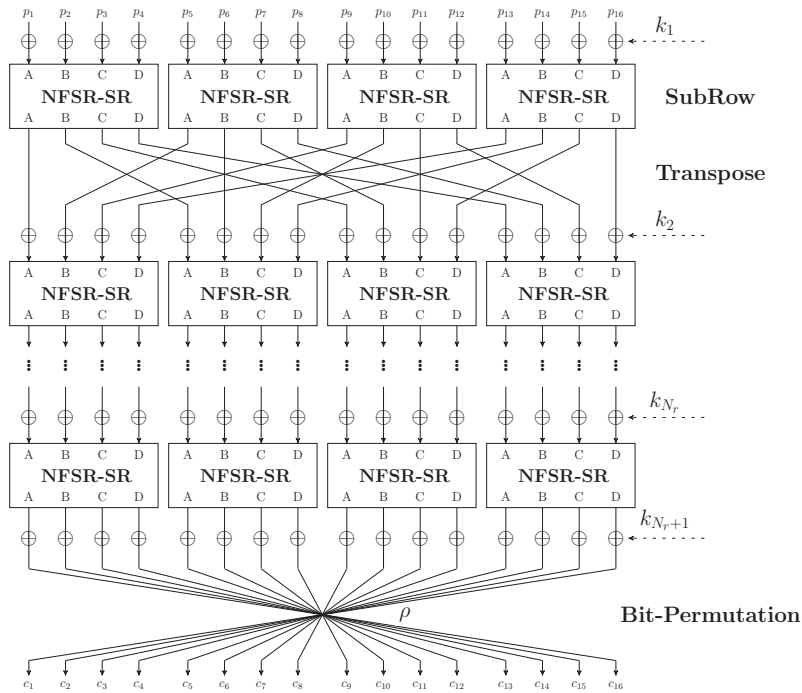


图 1 SPRING-128-128 加密算法示意图

Figure 1 An overview of SPRING-128-128 encryption algorithm

那么 AddRoundKey 的计算规则为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix} \oplus \begin{pmatrix} k_{11} & k_{12} & k_{13} & k_{14} \\ k_{21} & k_{22} & k_{23} & k_{24} \\ k_{31} & k_{32} & k_{33} & k_{34} \\ k_{41} & k_{42} & k_{43} & k_{44} \end{pmatrix} = \begin{pmatrix} s'_{11} & s'_{12} & s'_{13} & s'_{14} \\ s'_{21} & s'_{22} & s'_{23} & s'_{24} \\ s'_{31} & s'_{32} & s'_{33} & s'_{34} \\ s'_{41} & s'_{42} & s'_{43} & s'_{44} \end{pmatrix}$$

其中 $s'_{ij} = s_{ij} \oplus k_{ij}, 1 \leq i, j \leq 4$.

2.2.2 SubRow 的定义

SubRow 是 $\mathbb{F}_2^{32}$ 到 $\mathbb{F}_2^{32}$ 的映射, 对内部状态矩阵的一行做非线性变换, 参见图 2, 是算法中唯一的非线性运算. 注意到存储一个 $\mathbb{F}_2^{32}$ 到 $\mathbb{F}_2^{32}$ 的 S-盒需要占用很大的内存空间, SubRow 实际是由一个 32 比特非线性反馈移位寄存器 (NFSR) 来实现的, 称为 NFSR-SR, 其中 NFSR-SR 包含 4 个 8 比特寄存器 A、B、C、D, 具体设计参数请参见第 2.2.3 节. 设 SubRow 运算的输入为 $s_{i1}, s_{i2}, s_{i3}, s_{i4}$, 即内部状态的第 i 行, $1 \leq i \leq 4$, 则用 $s_{i1}, s_{i2}, s_{i3}, s_{i4}$ 分别填充寄存器 A、B、C、D, 然后 NFSR-SR 更新 32 拍, 直接输出内部状态 $s'_{i1}, s'_{i2}, s'_{i3}, s'_{i4}$, 参见图 3.

2.2.3 NFSR-SR 设计参数

NFSR-SR 是一个 32 比特的非线性反馈移位寄存器, 可看作 4 个 8-bit 寄存器 A、B、C、D 互相反馈的环状串联结构, 如图 4 所示, 其中 4 个寄存器 A、B、C、D 的反馈函数分别为

$$f_A = x_6 \oplus x_4 x_3 \oplus x_0,$$

$$f_B = x_5 \oplus x_4 x_3 \oplus x_0,$$

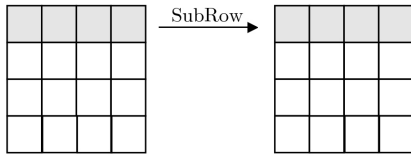


图 2 SubRow 示意图
Figure 2 Overview of SubRow

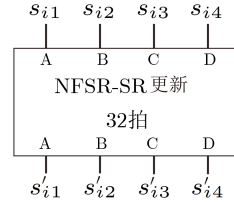


图 3 SubRow($s_{i1}, s_{i2}, s_{i3}, s_{i4}$) 示意图
Figure 3 SubRow($s_{i1}, s_{i2}, s_{i3}, s_{i4}$) with an input

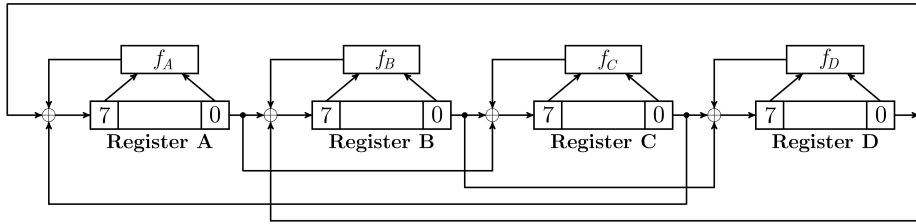


图 4 NFSR-SR 示意图
Figure 4 An overview of NFSR-SR

$$f_C = x_4 x_5 \oplus x_3 \oplus x_0,$$

$$f_D = x_4 x_5 \oplus x_2 \oplus x_0$$

为了描述的清晰性, 下面给出 32 比特 NFSR-SR 的状态转移函数. 设 t 时刻 NFSR-SR 的内部状态为

$$\begin{aligned} S(t) &= (A(t), B(t), C(t), D(t)) \\ &= (a_7(t), \dots, a_0(t); b_7(t), \dots, b_0(t); c_7(t), \dots, c_0(t); d_7(t), \dots, d_0(t)) \end{aligned}$$

则第 $t+1$ 时刻的状态为

$$\begin{aligned} a_7(t+1) &= a_6(t) \oplus a_4(t) a_3(t) \oplus a_0(t) \oplus c_0(t) \oplus d_0(t), \\ a_i(t+1) &= a_{i+1}(t), i = 6, 5, \dots, 0, \\ b_7(t+1) &= b_5(t) \oplus b_4(t) b_3(t) \oplus b_0(t) \oplus a_0(t) \oplus d_0(t), \\ b_i(t+1) &= b_{i+1}(t), i = 6, 5, \dots, 0, \\ c_7(t+1) &= c_4(t) c_5(t) \oplus c_3(t) \oplus c_0(t) \oplus b_0(t) \oplus a_0(t), \\ c_i(t+1) &= c_{i+1}(t), i = 6, 5, \dots, 0, \\ d_7(t+1) &= d_4(t) d_5(t) \oplus d_2(t) \oplus d_0(t) \oplus c_0(t) \oplus b_0(t), \\ d_i(t+1) &= d_{i+1}(t), i = 6, 5, \dots, 0 \end{aligned}$$

可以证明 NFSR-SR 的内部状态更新函数是可逆的.

命题 1 非线性反馈移位寄存器 NFSR-SR 的内部状态更新函数是可逆的.

证明: 这里我们直接给出状态更新函数的逆映射. 设已知第 $t+1$ 时刻内部状态 $S(t+1) = (A(t+1), B(t+1), C(t+1), D(t+1))$, 并记

$$\begin{aligned} \alpha &= a_7(t+1) \oplus a_5(t+1) \oplus a_3(t+1) a_2(t+1) \\ \beta &= b_7(t+1) \oplus b_4(t+1) \oplus b_3(t+1) b_2(t+1) \end{aligned}$$

$$\begin{aligned}\gamma &= c_7(t+1) \oplus c_3(t+1) c_4(t+1) \oplus c_2(t+1) \\ \theta &= d_7(t+1) \oplus d_3(t+1) d_4(t+1) \oplus d_1(t+1)\end{aligned}$$

下面给出第 t 时刻内部状态 $S(t)$ 的计算公式

$$\begin{aligned}a_0(t) &= \alpha \oplus \beta \oplus \gamma, \quad a_i(t) = a_{i-1}(t+1), i = 1, 2, \dots, 7 \\ b_0(t) &= \beta \oplus \gamma \oplus \theta, \quad b_i(t) = b_{i-1}(t+1), i = 1, 2, \dots, 7 \\ c_0(t) &= \alpha \oplus \gamma \oplus \theta, \quad c_i(t) = c_{i-1}(t+1), i = 1, 2, \dots, 7 \\ d_0(t) &= \alpha \oplus \beta \oplus \theta, \quad d_i(t) = d_{i-1}(t+1), i = 1, 2, \dots, 7\end{aligned}$$

□

由 NFSR-SR 更新函数的可逆性易知下述命题 2 成立.

命题 2 基于 NFSR-SR 的 SubRow 函数是 $\mathbb{F}_2^{32}$ 到 $\mathbb{F}_2^{32}$ 的一一映射.

2.2.4 Transpose 的定义

Transpose 对内部状态矩阵进行一次转置, 即

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix} \xrightarrow{\text{Transpose}} \begin{pmatrix} s_{11} & s_{21} & s_{31} & s_{41} \\ s_{12} & s_{22} & s_{32} & s_{42} \\ s_{13} & s_{23} & s_{33} & s_{43} \\ s_{14} & s_{24} & s_{34} & s_{44} \end{pmatrix}$$

2.2.5 BitPermutation 的定义

BitPermutation 仅用在最后一轮轮函数 FinalRound 中, 其功能是对内部状态的比特位置进行一次翻转. 设 $(a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0)$ 是一个字节的 8 比特, 其中 a_7 是高位比特, 记

$$\rho(a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0) = (a_0 a_1 a_2 a_3 a_4 a_5 a_6 a_7)$$

即 ρ 表示翻转一个字节. 基于作用在字节上的 ρ 函数, BitPermutation 的定义如下

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix} \xrightarrow{\text{Bit-Permutation}} \begin{pmatrix} \rho(s_{44}) & \rho(s_{43}) & \rho(s_{42}) & \rho(s_{41}) \\ \rho(s_{34}) & \rho(s_{33}) & \rho(s_{32}) & \rho(s_{31}) \\ \rho(s_{24}) & \rho(s_{23}) & \rho(s_{22}) & \rho(s_{21}) \\ \rho(s_{14}) & \rho(s_{13}) & \rho(s_{12}) & \rho(s_{11}) \end{pmatrix}$$

2.2.6 KeyExpansion 的设计

本小节介绍 SPRING-128-128 加密算法中的密钥扩展算法. 设私钥为 $\text{key} = (k_0, k_1, \dots, k_{15})$, 其中 k_i 是一个字节, 即 8 比特. 轮子密钥由一个 128 比特的非线性反馈移位寄存器生成, 称为 NFSR-KeyGeneration. 该 NFSR-KeyGeneration 是 8 个 16 比特寄存器的环状串联结构, 如图 5 所示, 其中 8 个寄存器的反馈函数分别为

$$\begin{aligned}f_1 &= x_1 \oplus x_3 \oplus x_8 x_9, & f_5 &= x_{15} \oplus x_7 x_8 \oplus x_{10} \\ f_2 &= x_1 \oplus x_4 \oplus x_8 x_9, & f_6 &= x_{15} \oplus x_7 x_8 \oplus x_{11} \\ f_3 &= x_1 \oplus x_5 \oplus x_8 x_9, & f_7 &= x_{15} \oplus x_7 x_8 \oplus x_{12} \\ f_4 &= x_1 \oplus x_6 \oplus x_8 x_9, & f_8 &= x_{15} \oplus x_7 x_8 \oplus x_{13}\end{aligned}$$

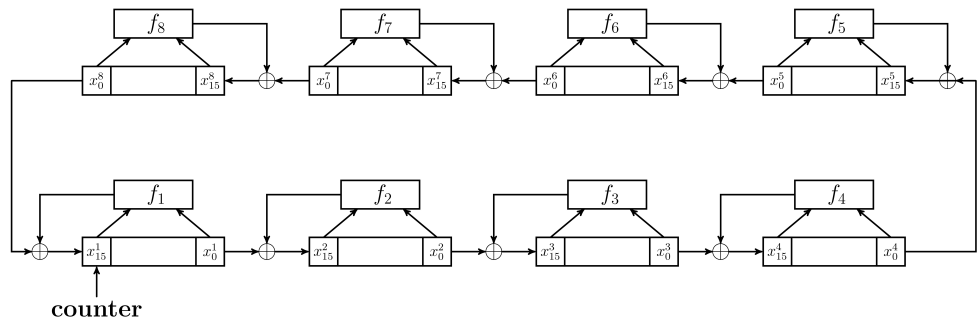


图 5 非线性反馈移位寄存器 NFSR-KeyGeneration 示意图
Figure 5 Overview of NFSR-KeyGeneration

设图 5 中 NFSR-KeyGeneration 的内部状态为 $IS = (IS_1^1, IS_2^1, IS_1^2, IS_2^2, \dots, IS_1^8, IS_2^8)$, 其中 (IS_1^i, IS_2^i) 是寄存器 i 的内部状态, 并且 $(IS_1^i, IS_2^i) = (x_{15}^i, x_{14}^i, \dots, x_0^i)$. 具体的轮子密钥生成算法参见算法 5, 其中常值 counter 的取值参见表 1. 常值 counter 可以由一个 8 比特的 LFSR 生成, 特征多项式为 $f(y) = y^8 + y^4 + y^3 + y^2 + 1$, 每隔 8 拍取内部状态为 counter 值.

算法 5 KeyExpansion-128-128	
Input: key	
Output: $(rk_1, rk_2, \dots, rk_{N_r+1})$	
1	$IS = (IS_1^1, IS_2^1, IS_1^2, IS_2^2, \dots, IS_1^8, IS_2^8) \leftarrow (k_0, k_1, \dots, k_{15});$
2	$rk_1 \leftarrow IS;$
3	for $2 \leq i \leq N_r + 1$ do
4	$IS_1^1 \leftarrow IS_1^1 \oplus \text{counter}[i - 1];$
5	NFSR-KeyGeneration 运行 16 拍;
6	$rk_i \leftarrow IS;$
7	end

注: 若记 $\text{counter}[i - 1] = (y_7, y_6, \dots, y_0)$, 则 $IS_1^1 \oplus \text{counter}[i - 1] = (x_{15}^1 \oplus y_7, x_{14}^1 \oplus y_6, \dots, x_8^1 \oplus y_0)$.

表 1 SPRING-128-128 密钥扩展算法中的常值
Table 1 Counter values in SPRING-128-128

常数	值	常数	值
counter[1]	(1, 1, 0, 0, 0, 0, 0, 0)	counter[6]	(0, 1, 0, 0, 1, 1, 0, 1)
counter[2]	(0, 0, 1, 0, 0, 1, 0, 0)	counter[7]	(1, 1, 0, 0, 0, 0, 1, 1)
counter[3]	(0, 0, 1, 1, 1, 0, 1, 1)	counter[8]	(1, 0, 1, 1, 0, 1, 1, 1)
counter[4]	(0, 1, 0, 0, 0, 0, 0, 1)	counter[9]	(1, 1, 0, 1, 0, 1, 1, 1)
counter[5]	(0, 1, 1, 0, 1, 1, 0, 1)	counter[10]	(0, 1, 0, 0, 0, 1, 0, 1)

2.3 SPRING-128-128 解密算法

SPRING-128-128 解密算法与加密算法具有相似性. SPRING-128-128 解密算法的轮函数 InvRound-128 参见算法 6, 最后一轮轮函数 InvFinalRound-128 参见算法 7.

算法 6 InvRound-128	算法 7 InvFinalRound-128
Input: state, rk_i^{inv} 1 AddRoundKey(state, rk_i^{inv}); 2 for $1 \leq j \leq 4$ do 3 InvSubRow($s_{j1}, s_{j2}, s_{j3}, s_{j4}$); 4 end 5 Transpose(state);	Input: state, $rk_{N_r}^{\text{inv}}, rk_{N_r+1}^{\text{inv}}$ 1 AddRoundKey(state, $rk_{N_r}^{\text{inv}}$); 2 for $1 \leq j \leq 4$ do 3 InvSubRow($s_{j1}, s_{j2}, s_{j3}, s_{j4}$); 4 end 5 AddRoundKey(state, $rk_{N_r+1}^{\text{inv}}$); 6 BitPermutation(state);

2.3.1 InvSubRow 与 NFSR-SR-Inv 的设计

与 SubRow 类似, InvSubRow 是 $\mathbb{F}_2^{32}$ 到 $\mathbb{F}_2^{32}$ 的映射, 对内部状态矩阵的一行做非线性变换. InvSubRow 由一个 32 比特的非线性反馈移位寄存器 NFSR-SR-Inv 来实现, 参见图 6. 设 InvSubRow 运算的输入为 $s_{i1}, s_{i2}, s_{i3}, s_{i4}$, 即内部状态的第 i 行, $1 \leq i \leq 4$, 则用 $s_{i1}, s_{i2}, s_{i3}, s_{i4}$ 分别填充寄存器 A、B、C、D, 然后 NFSR-SR-Inv 更新 32 拍, 直接输出内部状态 $s'_{i1}, s'_{i2}, s'_{i3}, s'_{i4}$, 参见图 7.

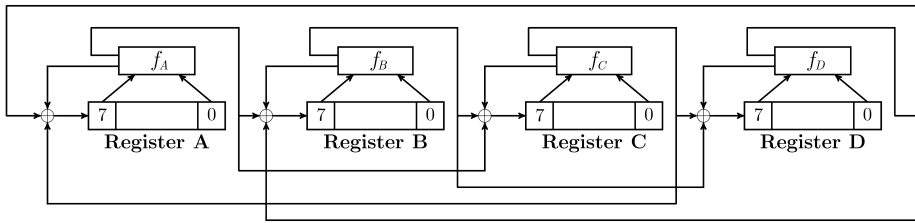


图 6 非线性反馈移位寄存器 NFSR-SR-Inv 示意图

Figure 6 Overview of NFSR-SR-Inv

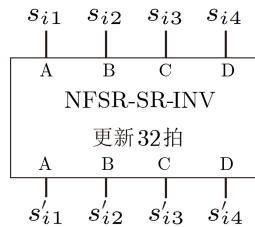


图 7 NFSR-SR-Inv($s_{i1}, s_{i2}, s_{i3}, s_{i4}$) 示意图

Figure 7 Overview of NFSR-SR-Inv($s_{i1}, s_{i2}, s_{i3}, s_{i4}$)

NFSR-SR-Inv 仍然是 4 个 8 比特寄存器 A、B、C、D 的环状串联, 4 个寄存器的反馈函数 f_A, f_B, f_C 和 f_D 参见第 2.2.3 节, 也即与 SubRow 中 4 个寄存器的反馈函数是一样的. NFSR-SR-Inv 的工作原理参见图 6. 事实上 NFSR-SR-Inv 与 NFSR-SR 可以用同一个电路来实现, 仅需额外增加一个选择器, 参见图 8.

InvSubRow 与 SubRow 具有以下逆映射关系.

命题 3 若 SubRow 将 $\alpha = (\alpha_A, \alpha_B, \alpha_C, \alpha_D) \in \mathbb{F}_2^{32}$ 映射到 $\beta = (\beta_A, \beta_B, \beta_C, \beta_D) \in \mathbb{F}_2^{32}$, 则 InvSubRow 将 $\text{BitPermutation}(\beta) = (\rho(\beta_D), \rho(\beta_C), \rho(\beta_B), \rho(\beta_A))$ 映射到 $\text{BitPermutation}(\alpha) = (\rho(\alpha_D), \rho(\alpha_C), \rho(\alpha_B), \rho(\alpha_A))$, 也即函数 $\text{BitPermutation} \circ \text{InvSubRow} \circ \text{BitPermutation}$ 是 SubRow 的逆函数.

2.3.2 InvKeyExpansion 的设计

若在加密时, 存储了所有的轮子密钥 $(rk_1, rk_2, \dots, rk_{N_r+1})$, 则在解密时对每个子密钥做一个简单的线性变换即可得到解密轮子密钥, 参见算法 8.

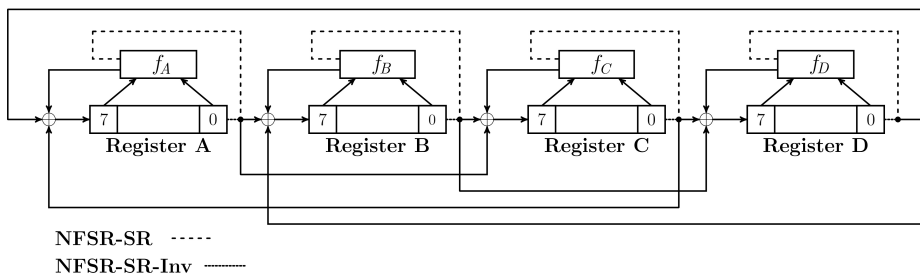


图 8 NFSR-SR 与 NFSR-SR-Inv 统一示意图
Figure 8 NFSR-SR and NFSR-SR-Inv

注 1 为了保持解密算法与加密算法结构的一致性, 我们在算法 8 中交换了轮密钥加和 Transpose 运算的顺序.

算法 8 InvKeyExpansion-RK-128

Input: $rk_1, rk_2, \dots, rk_{N_r+1}$
Output: $(rk_1^{inv}, rk_2^{inv}, \dots, rk_{N_r+1}^{inv})$
 1 $rk_1^{inv} \leftarrow \text{BitPermutation}(rk_{N_r+1});$
 2 **for** $2 \leq i \leq N_r$ **do**
 3 $rk_i^{inv} \leftarrow \text{Transpose}(\text{BitPermutation}(rk_{N_r-i+2}));$
 4 **end**
 5 $rk_{N_r+1}^{inv} \leftarrow \text{BitPermutation}(rk_1);$

若不存储加密算法使用的所有轮子密钥, 也可以通过加密算法的最后一轮轮子密钥重新生成解密轮子密钥. 设加密算法中最后一轮轮子密钥为 $\text{FinalKey} = (FK_1, FK_2, \dots, FK_{16})$, 其中 FK_i 是一个字节. 解密轮子密钥可由一个 128 比特的非线性反馈移位寄存器生成, 称为 NFSR-KeyGeneration-Inv, 如图 9, 其中反馈函数 $f_1, f_2, \dots, f_8$ 的定义参见第 2.2.6 节, 也即与 NFSR-KeyGeneration 中 8 个寄存器的反馈函数是一样的.

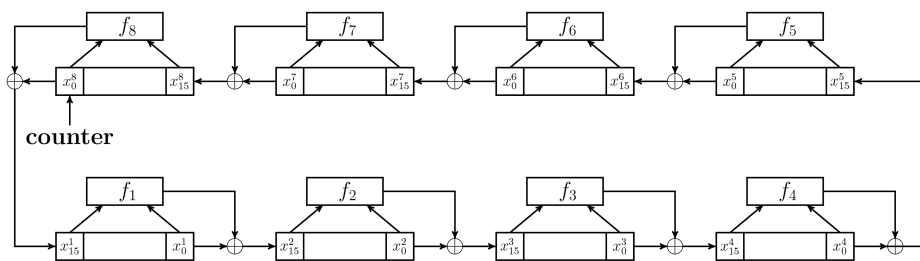


图 9 非线性反馈移位寄存器 NFSR-KeyGeneration-Inv 示意图
Figure 9 Overview of NFSR-KeyGeneration-Inv

设 NFSR-KeyGeneration-Inv 的内部状态为 $IS = (IS_1^1, IS_2^1, IS_2^2, IS_2^3, \dots, IS_1^8, IS_2^8)$, 其中 (IS_1^i, IS_2^i) 是寄存器 i 的内部状态, 并且 $(IS_1^i, IS_2^i) = (x_{15}^i, x_{14}^i, \dots, x_0^i)$. 基于非线性反馈移位寄存器 NFSR-KeyGeneration-Inv 的解密轮子密钥生成算法参见算法 9.

注 2 比较图 5 与图 9, 易见 NFSR-KeyGeneration-Inv 与 NFSR-KeyGeneration 仅有以下两点不同:

- (1) counter 值加入的位置不同;
- (2) 反馈函数输出的拉线方向不同.

因此, 在硬件设计时, 可以用同一个电路实现 NFSR-KeyGeneration-Inv 与 NFSR-KeyGeneration 的功能, 仅需额外增加一个线路的选择器.

算法 9 InvKeyExpansion-FK-128-128

Input: FinalKey
Output: $(rk_1^{inv}, rk_2^{inv}, \dots, rk_{N_r+1}^{inv})$

- 1 $IS = (IS_1^1, IS_2^1, IS_3^1, IS_4^1, \dots, IS_1^8, IS_2^8) \leftarrow (\rho(FK_{16}), \rho(FK_{15}), \dots, \rho(FK_1))$;
- 2 $rk_1^{inv} \leftarrow IS$;
- 3 **for** $2 \leq i \leq N_r$ **do**
- 4 NFSR-KeyGeneration-Inv 运行 16 拍;
- 5 $IS_2^8 \leftarrow IS_2^8 \oplus \rho(\text{counter}[N_r - i + 2])$;
- 6 $rk_i^{inv} \leftarrow \text{Transpose}(IS)$;
- 7 **end**
- 8 NFSR-KeyGeneration-Inv 运行 16 拍;
- 9 $IS_2^8 \leftarrow IS_2^8 \oplus \rho(\text{counter}[1])$;
- 10 $rk_{N_r+1}^{inv} \leftarrow IS$;

3 SPRING-128-256 的设计

SPRING-128-256 与 SPRING-128-128 的加/解密算法是一样的, 不同之处仅在于轮数和密钥扩展算法. SPRING-128-256 的迭代轮数为 14 轮. 以下对 SPRING-128-256 的密钥扩展算法进行详细描述.

3.1 KeyExpansion 的设计

设私钥为 $\text{key} = (k_1, k_2, \dots, k_{32})$, 其中 k_i 是一个字节, $1 \leq i \leq 32$. SPRING-128-256 的轮子密钥由一个 256 比特的非线性反馈移位寄存器生成, 称为 NFSR-KeyGeneration-256. 该非线性反馈移位寄存器可以看成是 8 个 32 比特寄存器的环状串联结构, 如图 10 所示, 其中 8 个寄存器的反馈函数分别为

$$\begin{aligned} g_1 &= x_1 \oplus x_5 \oplus x_{15}x_{16}, & g_5 &= x_{31} \oplus x_{16}x_{17} \oplus x_{24}, \\ g_2 &= x_1 \oplus x_6 \oplus x_{15}x_{16}, & g_6 &= x_{31} \oplus x_{16}x_{17} \oplus x_{25}, \\ g_3 &= x_1 \oplus x_7 \oplus x_{15}x_{16}, & g_7 &= x_{31} \oplus x_{16}x_{17} \oplus x_{26}, \\ g_4 &= x_1 \oplus x_8 \oplus x_{15}x_{16}, & g_8 &= x_{31} \oplus x_{16}x_{17} \oplus x_{27} \end{aligned}$$

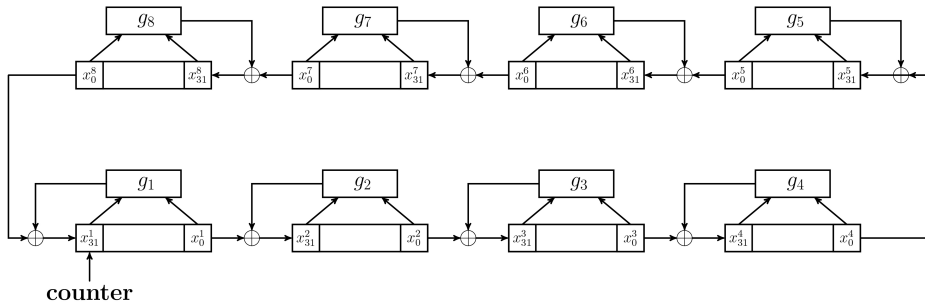


图 10 非线性反馈移位寄存器 NFSR-KeyGeneration-256 示意图
 Figure 10 Overview of NFSR-KeyGeneration-256

设图 10 中 NFSR-KeyGeneration-256 的内部状态为

$$IS = (IS_1^1, IS_2^1, IS_3^1, IS_4^1, IS_1^2, IS_2^2, IS_3^2, IS_4^2, \dots, IS_1^8, IS_2^8, IS_3^8, IS_4^8)$$

其中 $(IS_1^i, IS_2^i, IS_3^i, IS_4^i)$ 是寄存器 i 的内部状态, 并且 $(IS_1^i, IS_2^i, IS_3^i, IS_4^i) = (x_{31}^i, x_{30}^i, \dots, x_0^i)$. 具体的轮子密钥生成算法参见算法 10. 常值 counter 可以由一个 8 比特的 LFSR 生成, 特征多项式为 $f(y) = y^8 + y^4 + y^3 + y^2 + 1$, 每隔 8 拍取内部状态为 counter 值, 具体取值参见表 2.

算法 10 KeyExpansion-128-256

Input: key**Output:** $(rk_1, rk_2, \dots, rk_{N_r+1})$

```

1 IS =  $(IS_1^1, IS_2^1, IS_3^1, IS_4^1, \dots, IS_1^8, IS_2^8, IS_3^8, IS_4^8) \leftarrow (k_1, k_2, \dots, k_{32})$ ;
2  $rk_1 \leftarrow (IS^1, IS^3, IS^5, IS^7)$ , 其中  $IS^i = (IS_1^i, IS_2^i, IS_3^i, IS_4^i), i = 1, 3, 5, 7$ ;
3 for  $2 \leq i \leq N_r + 1$  do
4    $IS_1^1 \leftarrow IS_1^1 \oplus \text{counter}[i - 1]$ ;
5   NFSR-KeyGeneration-256 运行 32 拍;
6    $rk_i \leftarrow (IS^1, IS^3, IS^5, IS^7)$ ;
7 end

```

表 2 SPRING-128-256 密钥扩展算法中的常值
Table 2 Counter values in SPRING-128-256

常数	值	常数	值
counter[1]	(1, 1, 0, 0, 0, 0, 0, 0)	counter[8]	(1, 0, 1, 1, 0, 1, 1, 1)
counter[2]	(0, 0, 1, 0, 0, 1, 0, 0)	counter[9]	(1, 1, 0, 1, 0, 1, 1, 1)
counter[3]	(0, 0, 1, 1, 1, 0, 1, 1)	counter[10]	(0, 1, 0, 0, 0, 1, 0, 1)
counter[4]	(0, 1, 0, 0, 0, 0, 0, 1)	counter[11]	(1, 1, 0, 1, 1, 0, 0, 0)
counter[5]	(0, 1, 1, 0, 1, 1, 0, 1)	counter[12]	(0, 1, 1, 1, 1, 0, 0, 0)
counter[6]	(0, 1, 0, 0, 1, 1, 0, 1)	counter[13]	(1, 1, 0, 0, 1, 1, 1, 0)
counter[7]	(1, 1, 0, 0, 0, 0, 1, 1)	counter[14]	(0, 1, 1, 0, 1, 0, 0, 0)

3.2 InvKeyExpansion 的设计

若在加密时, 存储了所有轮子密钥 $(rk_1, rk_2, \dots, rk_{N_r+1})$, 则 SPRING-128-256 算法的解密密钥仅需对每个子密钥做一个简单的线性变换即可得到, 即算法 8, 其中轮数 $N_r = 14$.

若不存储加密算法使用的所有轮子密钥, 可以通过加密算法的最后一轮轮子密钥重新生成解密轮子密钥. 设最后一轮轮子密钥为 $\text{FinalKey} = (FK_1, FK_2, \dots, FK_{32})$, 其中 FK_i 是一个字节, $1 \leq i \leq 32$. 解密轮子密钥由一个 256 比特的非线性反馈移位寄存器生成, 称为 NFSR-KeyGeneration-Inv-256, 如图 11 所示, 其中 8 个寄存器的反馈函数 $g_1, g_2, \dots, g_8$ 与 NFSR-KeyGeneration-256 中是一样的. 易见, NFSR-KeyGeneration-Inv-256 和 NFSR-KeyGeneration-256 差别不大, 用同一个电路实现 NFSR-KeyGeneration-Inv-256 与 NFSR-KeyGeneration-256 只需要额外增加极少的硬件资源.

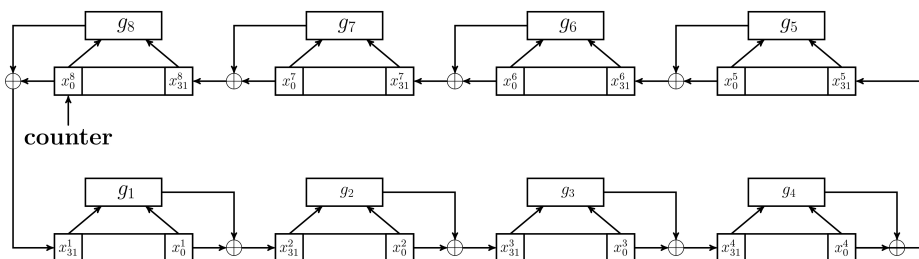


图 11 非线性反馈移位寄存器 NFSR-KeyGeneration-Inv-256 示意图

Figure 11 Overview of NFSR-KeyGeneration-Inv-256

设图 11 中 NFSR-KeyGeneration-Inv-256 的内部状态为

$$IS = (IS_1^1, IS_2^1, IS_3^1, IS_4^1, IS_1^2, IS_2^2, IS_3^2, IS_4^2, \dots, IS_1^8, IS_2^8, IS_3^8, IS_4^8)$$

其中 $(IS_1^i, IS_2^i, IS_3^i, IS_4^i)$ 是寄存器 i 的内部状态, 并且 $(IS_1^i, IS_2^i, IS_3^i, IS_4^i) = (x_{31}^i, x_{30}^i, \dots, x_0^i)$. 解密轮子密钥生成算法参见算法 11.

算法 11 InvKeyExpansion-FK-128-256

Input: FinalKey
Output: $(rk_1^{\text{inv}}, rk_2^{\text{inv}}, \dots, rk_{N_r+1}^{\text{inv}})$

- 1 $IS = (IS_1^1, IS_2^1, IS_3^1, IS_4^1, \dots, IS_1^8, IS_2^8, IS_3^8, IS_4^8) \leftarrow (\rho(FK_{32}), \rho(FK_{31}), \dots, \rho(FK_1));$
- 2 $rk_1^{\text{inv}} \leftarrow (IS^2, IS^4, IS^6, IS^8)$, 其中 $IS^i = (IS_1^i, IS_2^i, IS_3^i, IS_4^i), i = 2, 4, 6, 8;$
- 3 **for** $2 \leq i \leq N_r$ **do**
- 4 NFSR-KeyGeneration-Inv-256 运行 32 拍;
- 5 $IS_4^8 \leftarrow IS_4^8 \oplus \rho(\text{counter}[N_r - i + 2]);$
- 6 $rk_i^{\text{inv}} \leftarrow \text{Transpose}(IS^2, IS^4, IS^6, IS^8);$
- 7 **end**
- 8 NFSR-KeyGeneration-Inv-256 运行 32 拍;
- 9 $IS_4^8 \leftarrow IS_4^8 \oplus \rho(\text{counter}[1]);$
- 10 $rk_{N_r+1}^{\text{inv}} \leftarrow (IS^2, IS^4, IS^6, IS^8);$

4 SPRING256-256 的设计

本节介绍 SPRING-256-256 的设计参数.

4.1 SPRING-256-256 的内部状态

SPRING-256-256 的内部状态以字节为单位, 看作 $\mathbb{F}_{2^8}$ 上的 32 维向量或者 $\mathbb{F}_{2^8}$ 上 8×4 的矩阵, 即

$$\begin{aligned} \text{state} &= s_0 \parallel s_1 \parallel s_2 \parallel s_3 \parallel \dots \parallel s_{30} \parallel s_{31} \\ &= (s_0, s_1, s_2, s_3, \dots, s_{30}, s_{31}) \\ &= \begin{pmatrix} s_0 & s_1 & s_2 & s_3 \\ s_4 & s_5 & s_6 & s_7 \\ s_8 & s_9 & s_{10} & s_{11} \\ s_{12} & s_{13} & s_{14} & s_{15} \\ s_{16} & s_{17} & s_{18} & s_{19} \\ s_{20} & s_{21} & s_{22} & s_{23} \\ s_{24} & s_{25} & s_{26} & s_{27} \\ s_{28} & s_{29} & s_{30} & s_{31} \end{pmatrix} \end{aligned}$$

其中 s_i 为一个字节长度, 即 8 比特, $0 \leq i \leq 31$. 以下内部状态矩阵总是记为

$$\text{state} = \begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \\ s_{51} & s_{52} & s_{53} & s_{54} \\ s_{61} & s_{62} & s_{63} & s_{64} \\ s_{71} & s_{72} & s_{73} & s_{74} \\ s_{81} & s_{82} & s_{83} & s_{84} \end{pmatrix}$$

设 256 比特的明文分组为 $\text{plaintext} = p_0 \parallel p_1 \parallel \dots \parallel p_{31}$, 其中 p_i 是一个字节, $0 \leq i \leq 31$, 则用明文填充内部状态的规则为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \\ s_{51} & s_{52} & s_{53} & s_{54} \\ s_{61} & s_{62} & s_{63} & s_{64} \\ s_{71} & s_{72} & s_{73} & s_{74} \\ s_{81} & s_{82} & s_{83} & s_{84} \end{pmatrix} \leftarrow \begin{pmatrix} p_0 & p_1 & p_2 & p_3 \\ p_4 & p_5 & p_6 & p_7 \\ p_8 & p_9 & p_{10} & p_{11} \\ p_{12} & p_{13} & p_{14} & p_{15} \\ p_{16} & p_{17} & p_{18} & p_{19} \\ p_{20} & p_{21} & p_{22} & p_{23} \\ p_{24} & p_{25} & p_{26} & p_{27} \\ p_{28} & p_{29} & p_{30} & p_{31} \end{pmatrix}$$

设一个 256 比特密文分组为 $\text{ciphertext} = c_0 \parallel c_1 \parallel \cdots \parallel c_{31}$, 其中 c_i 是一个字节, $0 \leq i \leq 31$, 则内部状态与输出密文分组的对应关系为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \\ s_{51} & s_{52} & s_{53} & s_{54} \\ s_{61} & s_{62} & s_{63} & s_{64} \\ s_{71} & s_{72} & s_{73} & s_{74} \\ s_{81} & s_{82} & s_{83} & s_{84} \end{pmatrix} \rightarrow \begin{pmatrix} c_0 & c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 & c_7 \\ c_8 & c_9 & c_{10} & c_{11} \\ c_{12} & c_{13} & c_{14} & c_{15} \\ c_{16} & c_{17} & c_{18} & c_{19} \\ c_{20} & c_{21} & c_{22} & c_{23} \\ c_{24} & c_{25} & c_{26} & c_{27} \\ c_{28} & c_{29} & c_{30} & c_{31} \end{pmatrix}$$

4.2 SPRING-256-256 加密算法

SPRING-256-256 的加/解密结构与 SPRING-128-128 和 SPRING-128-256 是一致的. SPRING-256-256 算法的迭代轮数为 18 轮, 即 $N_r = 18$. SPRING-256-256 加密算法轮函数 Round-256 包括三个子函数: AddRoundKey, SubRow 和 Transpose-256. 在每一轮轮变换中, 首先加轮子密钥, 然后对内部状态矩阵的每一行分别做 SubRow 运算, 最后对内部状态进行一次位置置换 Transpose-256, 参见算法 12.

算法 12 Round-256

Input: state, rk_i

```

1 AddRoundKey(state,  $\text{rk}_i$ );
2 for  $1 \leq j \leq 8$  do
3   | SubRow( $s_{j1}, s_{j2}, s_{j3}, s_{j4}$ );
4 end
5 Transpose-256(state);
```

SPRING-256-256 最后一轮轮函数 FinalRound-256 包括三个子函数: AddRoundKey, SubRow 和 BitPermutation, 其中在 SubRow 运算前后各做了一次 AddRoundKey, 参见算法 13.

算法 13 FinalRound-256

Input: state, $\text{rk}_{N_r}, \text{rk}_{N_r+1}$

```

1 AddRoundKey(state,  $\text{rk}_{N_r}$ );
2 for  $1 \leq j \leq 8$  do
3   | SubRow( $s_{j1}, s_{j2}, s_{j3}, s_{j4}$ );
4 end
5 AddRoundKey(state,  $\text{rk}_{N_r+1}$ );
6 BitPermutation(state);
```

4.2.1 AddRoundKey 定义

设 AddRoundKey 的输入内部状态为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \\ s_{51} & s_{52} & s_{53} & s_{54} \\ s_{61} & s_{62} & s_{63} & s_{64} \\ s_{71} & s_{72} & s_{73} & s_{74} \\ s_{81} & s_{82} & s_{83} & s_{84} \end{pmatrix}$$

输入轮子密钥为

$$\begin{pmatrix} k_{11} & k_{12} & k_{13} & k_{14} \\ k_{21} & k_{22} & k_{23} & k_{24} \\ k_{31} & k_{32} & k_{33} & k_{34} \\ k_{41} & k_{42} & k_{43} & k_{44} \\ k_{51} & k_{52} & k_{53} & k_{54} \\ k_{61} & k_{62} & k_{63} & k_{64} \\ k_{71} & k_{72} & k_{73} & k_{74} \\ k_{81} & k_{82} & k_{83} & k_{84} \end{pmatrix}$$

那么 AddRoundKey 的计算规则为

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \\ s_{51} & s_{52} & s_{53} & s_{54} \\ s_{61} & s_{62} & s_{63} & s_{64} \\ s_{71} & s_{72} & s_{73} & s_{74} \\ s_{81} & s_{82} & s_{83} & s_{84} \end{pmatrix} \oplus \begin{pmatrix} k_{11} & k_{12} & k_{13} & k_{14} \\ k_{21} & k_{22} & k_{23} & k_{24} \\ k_{31} & k_{32} & k_{33} & k_{34} \\ k_{41} & k_{42} & k_{43} & k_{44} \\ k_{51} & k_{52} & k_{53} & k_{54} \\ k_{61} & k_{62} & k_{63} & k_{64} \\ k_{71} & k_{72} & k_{73} & k_{74} \\ k_{81} & k_{82} & k_{83} & k_{84} \end{pmatrix} = \begin{pmatrix} s'_{11} & s'_{12} & s'_{13} & s'_{14} \\ s'_{21} & s'_{22} & s'_{23} & s'_{24} \\ s'_{31} & s'_{32} & s'_{33} & s'_{34} \\ s'_{41} & s'_{42} & s'_{43} & s'_{44} \\ s'_{51} & s'_{52} & s'_{53} & s'_{54} \\ s'_{61} & s'_{62} & s'_{63} & s'_{64} \\ s'_{71} & s'_{72} & s'_{73} & s'_{74} \\ s'_{81} & s'_{82} & s'_{83} & s'_{84} \end{pmatrix}$$

其中 $s'_{ij} = s_{ij} \oplus k_{ij}, 1 \leq i \leq 8, 1 \leq j \leq 4$.

4.2.2 Transpose-256 的定义

Transpose-256 对内部状态矩阵进行如下置换:

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \\ s_{51} & s_{52} & s_{53} & s_{54} \\ s_{61} & s_{62} & s_{63} & s_{64} \\ s_{71} & s_{72} & s_{73} & s_{74} \\ s_{81} & s_{82} & s_{83} & s_{84} \end{pmatrix} \xrightarrow{\text{Transpose-256}} \begin{pmatrix} s_{11} & s_{21} & s_{31} & s_{41} \\ s_{51} & s_{61} & s_{71} & s_{81} \\ s_{12} & s_{22} & s_{32} & s_{42} \\ s_{52} & s_{62} & s_{72} & s_{82} \\ s_{13} & s_{23} & s_{33} & s_{43} \\ s_{53} & s_{63} & s_{73} & s_{83} \\ s_{14} & s_{24} & s_{34} & s_{44} \\ s_{54} & s_{64} & s_{74} & s_{84} \end{pmatrix}$$

Transpose-256 的逆变换记为 Transpose-Inv-256, 即

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \\ s_{51} & s_{52} & s_{53} & s_{54} \\ s_{61} & s_{62} & s_{63} & s_{64} \\ s_{71} & s_{72} & s_{73} & s_{74} \\ s_{81} & s_{82} & s_{83} & s_{84} \end{pmatrix} \xrightarrow{\text{Transpose-Inv-256}} \begin{pmatrix} s_{11} & s_{31} & s_{51} & s_{71} \\ s_{12} & s_{32} & s_{52} & s_{72} \\ s_{13} & s_{33} & s_{53} & s_{73} \\ s_{14} & s_{34} & s_{54} & s_{74} \\ s_{21} & s_{41} & s_{61} & s_{81} \\ s_{22} & s_{42} & s_{62} & s_{82} \\ s_{23} & s_{43} & s_{63} & s_{83} \\ s_{24} & s_{44} & s_{64} & s_{84} \end{pmatrix}$$

4.2.3 BitPermutation-256 的定义

类似 128 比特分组的情形, BitPermutation 仅用在最后一轮轮函数 FinalRound 中, 其功能是对内部状态的比特位置进行一次翻转. BitPermutation 的定义如下.

$$\begin{pmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \\ s_{51} & s_{52} & s_{53} & s_{54} \\ s_{61} & s_{62} & s_{63} & s_{64} \\ s_{71} & s_{72} & s_{73} & s_{74} \\ s_{81} & s_{82} & s_{83} & s_{84} \end{pmatrix} \xrightarrow{\text{BitPermutation}} \begin{pmatrix} \rho(s_{84}) & \rho(s_{83}) & \rho(s_{82}) & \rho(s_{81}) \\ \rho(s_{74}) & \rho(s_{73}) & \rho(s_{72}) & \rho(s_{71}) \\ \rho(s_{64}) & \rho(s_{63}) & \rho(s_{62}) & \rho(s_{61}) \\ \rho(s_{54}) & \rho(s_{53}) & \rho(s_{52}) & \rho(s_{51}) \\ \rho(s_{44}) & \rho(s_{43}) & \rho(s_{42}) & \rho(s_{41}) \\ \rho(s_{34}) & \rho(s_{33}) & \rho(s_{32}) & \rho(s_{31}) \\ \rho(s_{24}) & \rho(s_{23}) & \rho(s_{22}) & \rho(s_{21}) \\ \rho(s_{14}) & \rho(s_{13}) & \rho(s_{12}) & \rho(s_{11}) \end{pmatrix}$$

4.2.4 KeyExpansion 的设计

设私钥为 $\text{key} = (k_1, k_2, \dots, k_{32})$, 其中 k_i 是一个字节, $1 \leq i \leq 32$. SPRING-256-256 的密钥扩展算法与 SPRING-128-256 的基本相同, 不同之处在于 SPRING-128-256 提取非线性反馈移位寄存器 NFSR-KeyGeneration-256 内部状态部分比特作为轮子密钥, 而 SPRING-256-256 提取非线性反馈移位寄存器 NFSR-KeyGeneration-256 整个内部状态作为轮子密钥, 具体参见算法 14, 其中常数 counter 参见表 3.

算法 14 KeyExpansion-256-256

Input: key
Output: $(\text{rk}_1, \text{rk}_2, \dots, \text{rk}_{N_r+1})$

- 1 $\text{IS} = (\text{IS}_1^1, \text{IS}_2^1, \text{IS}_3^1, \text{IS}_4^1, \dots, \text{IS}_1^8, \text{IS}_2^8, \text{IS}_3^8, \text{IS}_4^8) \leftarrow (k_1, k_2, \dots, k_{32});$
- 2 $\text{rk}_1 \leftarrow \text{IS};$
- 3 **for** $2 \leq i \leq N_r + 1$ **do**
- 4 $\text{IS}_1^1 \leftarrow \text{IS}_1^1 \oplus \text{counter}[i - 1];$
- 5 NFSR-KeyGeneration-256 运行 32 拍;
- 6 $\text{rk}_i \leftarrow \text{IS};$
- 7 **end**

4.3 SPRING-256-256 解密算法

4.3.1 SPRING-256-256 解密算法轮函数的设计

SPRING-256-256 解密算法结构与加密算法是完全一致的, 其中非线性变换使用 InvSubRow, 线性变换使用 Transpose-Inv-256, 具体描述参见算法 15 和算法 16. 需要注意的是算法 15 中函数 Transpose-Inv-256 是 Transpose-256 的逆变换.

表 3 SPRING-256-256 密钥扩展算法中的常值
Table 3 Counter values in SPRING-256-256

常数	值	常数	值
counter[1]	(1, 1, 0, 0, 0, 0, 0, 0)	counter[10]	(0, 1, 0, 0, 0, 1, 0, 1)
counter[2]	(0, 0, 1, 0, 0, 1, 0, 0)	counter[11]	(1, 1, 0, 1, 1, 0, 0, 0)
counter[3]	(0, 0, 1, 1, 1, 0, 1, 1)	counter[12]	(0, 1, 1, 1, 1, 0, 0, 0)
counter[4]	(0, 1, 0, 0, 0, 0, 0, 1)	counter[13]	(1, 1, 0, 0, 1, 1, 1, 0)
counter[5]	(0, 1, 1, 0, 1, 1, 0, 1)	counter[14]	(0, 1, 1, 0, 1, 0, 0, 0)
counter[6]	(0, 1, 0, 0, 1, 1, 0, 1)	counter[15]	(1, 0, 0, 0, 1, 0, 0, 1)
counter[7]	(1, 1, 0, 0, 0, 0, 1, 1)	counter[16]	(0, 1, 0, 1, 0, 0, 1, 0)
counter[8]	(1, 0, 1, 1, 0, 1, 1, 1)	counter[17]	(1, 0, 1, 1, 1, 0, 0, 1)
counter[9]	(1, 1, 0, 1, 0, 1, 1, 1)	counter[18]	(1, 0, 0, 1, 1, 0, 1, 1)

算法 15 InvRound-256

Input: state, k_i^{inv}
1 AddRoundKey(state, k_i^{inv});
2 **for** $1 \leq j \leq 8$ **do**
3 InvSubRow($s_{j1}, s_{j2}, s_{j3}, s_{j4}$);
4 **end**
5 Transpose-Inv-256(state);

算法 16 InvFinalRound-256

Input: state, $k_{N_r}^{\text{inv}}, k_{N_r+1}^{\text{inv}}$
1 AddRoundKey(state, $k_{N_r}^{\text{inv}}$);
2 **for** $1 \leq j \leq 8$ **do**
3 InvSubRow($s_{j1}, s_{j2}, s_{j3}, s_{j4}$);
4 **end**
5 AddRoundKey(state, $k_{N_r+1}^{\text{inv}}$);
6 BitPermutation(state);

4.3.2 InvKeyExpansion 的设计

SPRING-256-256 解密算法中的密钥扩展算法与 SPRING-128-256 是基本相同的, 不同之处仅在于轮子密钥个数不同、每次提取非线性反馈移位寄存器 NFSR-KeyGeneration-Inv-256 内部状态比特数量不同.

若在加密过程中, 存储了所有轮子密钥 ($\text{rk}_1, \text{rk}_2, \dots, \text{rk}_{N_r+1}$), 则使用算法 17 来生成解密轮子密钥, 其中 $N_r = 18$.

算法 17 InvKeyExpansion-RK-256

Input: $\text{rk}_1, \text{rk}_2, \dots, \text{rk}_{N_r+1}$
Output: ($\text{rk}_1^{\text{inv}}, \text{rk}_2^{\text{inv}}, \dots, \text{rk}_{N_r+1}^{\text{inv}}$)
1 $\text{rk}_1^{\text{inv}} \leftarrow \text{BitPermutation}(\text{rk}_{N_r+1})$;
2 **for** $2 \leq i \leq N_r$ **do**
3 $\text{rk}_i^{\text{inv}} \leftarrow \text{Transpose-Inv-256}(\text{BitPermutation}(\text{rk}_{N_r-i+2}))$;
4 **end**
5 $\text{rk}_{N_r+1}^{\text{inv}} \leftarrow \text{BitPermutation}(\text{rk}_1)$;

若不存储加密算法使用的所有轮子密钥, 则可以通过加密算法的最后一轮轮子密钥 $\text{FinalKey} = (\text{FK}_1, \text{FK}_2, \dots, \text{FK}_{32})$ 按照算法 18 重新生成解密轮子密钥, 其中 FK_i 是一个字节, $1 \leq i \leq 32$.

算法 18 InvKeyExpansion-FK-256-256

Input: FinalKey
Output: $(rk_1^{inv}, rk_2^{inv}, \dots, rk_{N_r+1}^{inv})$

```

1 IS =  $(IS_1^1, IS_2^1, IS_3^1, IS_4^1, \dots, IS_1^8, IS_2^8, IS_3^8, IS_4^8) \leftarrow (\rho(FK_{32}), \rho(FK_{31}), \dots, \rho(FK_1))$ ;
2  $rk_1^{inv} \leftarrow IS$ ;
3 for  $2 \leq i \leq N_r$  do
4   NFSR-KeyGeneration-Inv-256 运行 32 拍;
5    $IS_4^8 \leftarrow IS_4^8 \oplus \rho(\text{counter}[N_r - i + 2])$ ;
6    $rk_i^{inv} \leftarrow \text{Transpose-Inv-256}(IS)$ ;
7 end
8 NFSR-KeyGeneration-Inv-256 运行 32 拍;
9  $IS_4^8 \leftarrow IS_4^8 \oplus \rho(\text{counter}[1])$ ;
10  $rk_{N_r+1}^{inv} \leftarrow IS$ ;
```

5 环状串联 NFSR 的性质

SPRING 算法中实现 S-盒功能的 NFSR-SR 和实现密钥扩展功能的 NFSR-KeyGeneration 均是环状串联非线性反馈移位寄存器。环状串联非线性反馈移位寄存器的概念最早在文献 [2] 中提出, 文中证明了环状串联非线性反馈移位寄存器中每个寄存器的输出序列具有相同的周期性质。NFSR-KeyGeneration 是典型的环状串联结构, 完全符合文献 [2] 的研究模型。NFSR-SR 是一个扩展的环状串联结构, 为了扩散更快, 在原文献 [2] 的研究模型基础上, 增加了四根对称的拉线, 依然具有类似环状串联的对称结构。

对于环状串联结构的 NFSR, 我们认为经过充分多的内部状态更新后, 每个比特寄存器具有非常等价的地位, 从而输出比特关于输入比特具有相似的密码学性质, 并且整体性强, 输入输出不容易分割成独立的小单元。

6 SubRow 运算的基本性质

在本节中, 我们对 SPRING 算法中唯一的非线性运算 SubRow 进行了分析和测试。SubRow 是 $\mathbb{F}_2^{32}$ 到 $\mathbb{F}_2^{32}$ 的函数, 实际由 32 比特的非线性反馈移位寄存器 NFSR-SR 实现。

6.1 代数性质

我们计算了 SubRow 运算每一个输出比特分位函数的真值表, 通过 Meobius 变换 [3] 分别计算出了 32 个输出比特分位函数的代数正规型。表 4 给出了每一个分位函数的代数次数和项数。观察表 4 可以发现, 各个输出比特分位函数的代数次数比较高并且项数稠密。

6.2 扩散性质

我们对非线性反馈移位寄存器 NFSR-SR 的扩散性质进行了测试。实验结果显示, 经过 25 轮迭代后, NFSR-SR 内部状态的每一比特与输入的每一比特相关, 也即每一状态比特函数的代数正规型包含 32 个输入比特变元。注意到在 SubRow 函数中 NFSR-SR 迭代 32 拍, 因此 SubRow 函数的每一输出比特与 32 个输入比特变量相关。

6.3 差分性质

注意到 SubRow 运算是一个 32 比特的非线性置换, 计算和存储整个差分分布表远远超出了普通计算机的计算能力。因此, 我们对 SubRow 运算的差分性质进行了以下测试。

首先, 我们利用太湖之光超级计算机穷尽了 SubRow 运算的差分分布表, 得到 SubRow 运算的最大差分概率等于 $\frac{152}{2^{32}} \approx 2^{-24.75}$, 即

$$\max_{\alpha, \beta \in \mathbb{F}_2^{32}, (\alpha, \beta) \neq (0, 0)} \#\{x \in \mathbb{F}_2^{32} : \text{SubRow}(x \oplus \alpha) \oplus \text{SubRow}(x) = \beta\} = 152$$

表 4 SubRow 运算各输出比特分位函数的代数次数和项数
Table 4 Algebraic degree and number of terms of each output bit in SubRow

比特分位函数	项数	代数次数	比特分位函数	项数	代数次数
A-7 分位	2145851065	28	C-7 分位	2147141457	28
A-6 分位	2140106291	27	C-6 分位	2145892641	28
A-5 分位	2110797249	26	C-5 分位	2139619927	27
A-4 分位	1997904877	24	C-4 分位	2110065801	26
A-3 分位	1686209527	23	C-3 分位	1990657315	25
A-2 分位	1206759085	22	C-2 分位	1704010563	23
A-1 分位	680494955	21	C-1 分位	1241057375	22
A-0 分位	316617067	19	C-0 分位	745924029	21
B-7 分位	2144409611	27	D-7 分位	2147440031	29
B-6 分位	2131110763	26	D-6 分位	2147027417	28
B-5 分位	2071634151	25	D-5 分位	2145285801	28
B-4 分位	1871941075	24	D-4 分位	2135753071	27
B-3 分位	1490247393	23	D-3 分位	2093209659	25
B-2 分位	982488275	21	D-2 分位	1946680355	24
B-1 分位	490884545	20	D-1 分位	1634756309	23
B-0 分位	219832871	18	D-0 分位	1111227209	21

并且有且仅有一对差分 (α, β) 取到上述最大值, 即

$$\alpha = 0x07060106, \beta = 0x509000f4$$

其中从高位到低位依次为寄存器

$$(A_7, A_6, \cdots, A_0, B_7, B_6, \cdots, B_0, C_7, C_6, \cdots, C_0, D_7, D_6, \cdots, D_0)$$

的差分.

其次, 对于 NFSR 的迭代模型, 相对于多重的输入差分, 一般单重输入差分扩更散慢. 例如在 Grain v1 的条件差分攻击中^[4, 5], 输入差分是单重差分. 于是, 我们计算了所有的单重输入差分对应的输出差分分布表. 表 5 给出了全体单重输入差分可以取到的最大差分概率.

表 5 单重输入差分的最大差分概率
Table 5 Maximal difference probability for input differences with one nonzero bit

差分位置	最大概率	差分位置	最大概率	差分位置	最大概率	差分位置	最大概率
A-8	$\frac{13}{2^{31}}$	B-8	$\frac{10}{2^{31}}$	C-8	$\frac{13}{2^{31}}$	D-8	$\frac{10}{2^{31}}$
A-7	$\frac{9}{2^{31}}$	B-7	$\frac{9}{2^{31}}$	C-7	$\frac{12}{2^{31}}$	D-7	$\frac{10}{2^{31}}$
A-6	$\frac{10}{2^{31}}$	B-6	$\frac{9}{2^{31}}$	C-6	$\frac{12}{2^{31}}$	D-6	$\frac{10}{2^{31}}$
A-5	$\frac{9}{2^{31}}$	B-5	$\frac{10}{2^{31}}$	C-5	$\frac{20}{2^{31}}$	D-5	$\frac{9}{2^{31}}$
A-4	$\frac{10}{2^{31}}$	B-4	$\frac{9}{2^{31}}$	C-4	$\frac{11}{2^{31}}$	D-4	$\frac{10}{2^{31}}$
A-3	$\frac{10}{2^{31}}$	B-3	$\frac{9}{2^{31}}$	C-3	$\frac{10}{2^{31}}$	D-3	$\frac{9}{2^{31}}$
A-2	$\frac{10}{2^{31}}$	B-2	$\frac{9}{2^{31}}$	C-2	$\frac{9}{2^{31}}$	D-2	$\frac{10}{2^{31}}$
A-1	$\frac{11}{2^{31}}$	B-1	$\frac{10}{2^{31}}$	C-1	$\frac{9}{2^{31}}$	D-1	$\frac{9}{2^{31}}$

最后, 因为 SPRING 算法的线性扩散层是字节拉线, 所以我们对单字节到单字节的差分概率进行了检测, 记录了各输入字节到各输出字节的最大差分概率, 见表 6.

表 6 输入字节到输出字节最大差分概率
Table 6 Maximal difference probability for byte-based differences

	第 1 字节	第 2 字节	第 3 字节	第 4 字节
第 1 字节	$\frac{6}{2^{31}}$	$\frac{5}{2^{31}}$	$\frac{5}{2^{31}}$	$\frac{6}{2^{31}}$
第 2 字节	$\frac{6}{2^{31}}$	$\frac{6}{2^{31}}$	$\frac{5}{2^{31}}$	$\frac{6}{2^{31}}$
第 3 字节	$\frac{7}{2^{31}}$	$\frac{6}{2^{31}}$	$\frac{5}{2^{31}}$	$\frac{6}{2^{31}}$
第 4 字节	$\frac{5}{2^{31}}$	$\frac{6}{2^{31}}$	$\frac{7}{2^{31}}$	$\frac{6}{2^{31}}$

7 硬件性能分析

SPRING 系列算法主要是面向硬件实现设计的. 根据不同的需求, 可以调节实现面积与加解密速率. 按轮实现, 面积最小; 全轮展开实现, 加解密速率最大. 我们用 Design Compiler 对 SPRING-128-128、SPRING-128-256、SPRING-256-256 的两种实现方式进行了综合, 其中工艺库为 TSMC 16 nm, 具体结果见表 7 和表 8.

表 7 单轮实现方式下 SPRING 算法的检测数据
Table 7 Test data for SPRING under round-based implementation

项目	128-128	128-256	256-256
加密周期	10 240	14 336	18 432
解密周期	10 400	14 784	19 008
关键路径时长 (ns)	0.171	0.1724	0.1679
时钟约束	5.6 GHz	5.6 GHz	5.6 GHz
加密速率 (Mbps)	2240	1600	2488.89
解密速率 (Mbps)	2205.54	1551.52	2413.47
面积 (μm^2)	1046.06	1468.93	2107.28
加密吞面比	2.1413	1.0892	1.1811
解密吞面比	2.1084	1.0562	1.1453

表 8 全轮实现方式下 SPRING 算法的检测数据
Table 8 Test data for SPRING under unrolled implementation

项目	128-128	128-256	256-256
加密周期	1312	1408	1568
解密周期	1472	1856	2144
关键路径时长 (ns)	0.1723	0.1858	0.1862
时钟约束	5.6 GHz	5.3 GHz	5.3 GHz
加密速率 (Mbps)	17 482.93	15 418.18	27 689.79
解密速率 (Mbps)	15 582.61	11 696.55	20 250.74
面积 (μm^2)	8079.41	15 107.54	29 195.68
加密吞面比	2.1639	1.0205	0.9484
解密吞面比	1.9287	0.7742	0.6936

8 总结

SPRING 是基于非线性反馈移位寄存器设计的轻量级分组密码。SPRING 采用 SP 结构, 非线性替换层通过 32 比特非线性反馈移位寄存器实现, 置换层是字节拉线, 加/解密对称, 整体结构简洁。用于 SPRING 替换层的 32 比特非线性反馈移位寄存器可以看作 32 进 32 出的可逆 S-盒, 而普通计算机无法穷尽其差分分布表和线性逼近概率表, 这使得传统的差分和线性分析方法很难应用到 SPRING 算法。目前, SPRING 算法最有效的差分分析方法是面向单字节的分析, 存在 4 轮单字节到单字节的有效差分特征。

References

- [1] WU W L, FENG D G, ZHANG W T. Design and Analysis of Block Ciphers[M]. 2nd Edition. Beijing: Tsinghua University Press, 2009.
吴文玲, 冯登国, 张文涛. 分组密码的设计与分析 [M]. 第 2 版. 北京: 清华大学出版社, 2009.
- [2] ZHAO X X, TIAN T, QI W F. A ring-like cascade connection and a class of NFSRs with the same cycle structures[J]. Designs, Codes and Cryptography, 2018: 86(12), 2775–2790. [DOI: 10.1007/s10623-018-0473-6]
- [3] JOUX A. Algorithmic Cryptanalysis[M]. CRC Press, 2009.
- [4] HELL M, JOHANSSON T, MAXIMOV A, et al. The Grain family of stream ciphers[M]. In: New Stream Cipher Designs—The eSTREAM Finalists. Springer Berlin Heidelberg, 2008: 179–190. [DOI: 10.1007/978-3-540-68351-3_14]
- [5] KNELLWOLF S, MEIER W, NAYA-PLASENCIA M. Conditional differential cryptanalysis of NLFSR-based cryptosystems[C]. In: Advances in Cryptology—ASIACRYPT 2010. Springer Berlin Heidelberg, 2010: 130–145. [DOI: 10.1007/978-3-642-17373-8_8]

作者信息

田甜 (1981–), 湖北武汉人, 副教授. 主要研究领域为对称密码设计与分析.
tiantian_d@126.com

戚文峰 (1963–), 浙江宁波人, 教授. 主要研究领域为对称密码设计与分析.
wenfeng.qi@263.net

叶晨东 (1992–), 浙江兰溪人, 博士生在读. 主要研究领域为序列密码算法的安全性分析.
ye_chendong@126.com

谢晓锋 (1993–), 福建厦门人, 博士生在读. 主要研究领域为分组密码算法的安全性分析.
xiaofengxie514@126.com